Verified Collaborative Editing with Mergeable Replicated Data Types

Anonymous Authors

Abstract

Collaborative rich-text editors need three guarantees at once: replicas must converge, their states must implement an ordinary sequential editor, and their metadata must remain collectable. We develop a Lean 4 verification of Peritext over mergeable replicated data types (MRDTs). The proof includes three sequence kernels: an RGA baseline, an immutable-coordinate EmbedRGA, and a sided EmbedRGA with machine-checked Fugue and FugueMax ordering-policy theorems. It proves replication convergence, local minted-history refinement to independent sequential editor machines, and full merged-history theorems for all three kernels. Two collectors address different sources of growth. An asynchronous distributed collector removes commit history; a Peritext-specific collector removes dead text records, deletion evidence, and closed mark pairs while preserving future edits. Their composition refines the uncollected virtual-merge-base semantics. Elementary arguments show that the retained metadata is necessary: a design that forgets deleted elements reorders survivors, admits no correct merge, and violates the strong list specification. A JavaScript implementation exercises the same designs. Repeated measurements show why both collectors and all three sequence baselines matter: on one long trace, EmbedRGA snapshots are 8.9 times smaller than RGA, while the sided variant pays additional metadata for a stronger non-interleaving policy.

Contents

1 The result **2**

2 Why plain RGA is the motivating example **3**
2.1 RGA . . . 3

3 Why sequence state must remember deleted elements **4**
3.1 Designs, diamonds, and specifications . . . 4
3.2 A forgetful delete reorders survivors on one replica . . . 5
3.3 No merge on forgetful states implements the RGA order . . . 6
3.4 The strong list specification forces the same memory . . . 6
3.5 How much must be remembered . . . 7
3.6 Convergence does not force retention . . . 8

4 Three verified sequence representations **8**
4.1 EmbedRGA . . . 8
4.2 SidedEmbedRGA and the Fugue policies . . . 9
4.3 What is proved for all three . . . 9

5 Peritext over the sided sequence **10**

6 Datatype-state garbage collection **11**
6.1 Three atomic collectors 12
6.2 Protocol refinement 12

7 Distributed commit-history garbage collection **13**

8 Ordering anomalies and evidence **14**

9 Implementation **14**

10 Evaluation **15**
10.1 Sequence kernels 15
10.2 Both collectors in Peritext 15
10.3 External plain-text systems 16

11 Evidence boundary and limitations **17**

12 Reproduction **17**

1 The result

The main result is an end-to-end semantic verification of a collaborative rich-text datatype. It is not one monolithic theorem. Figure 1 shows the evidence chain.

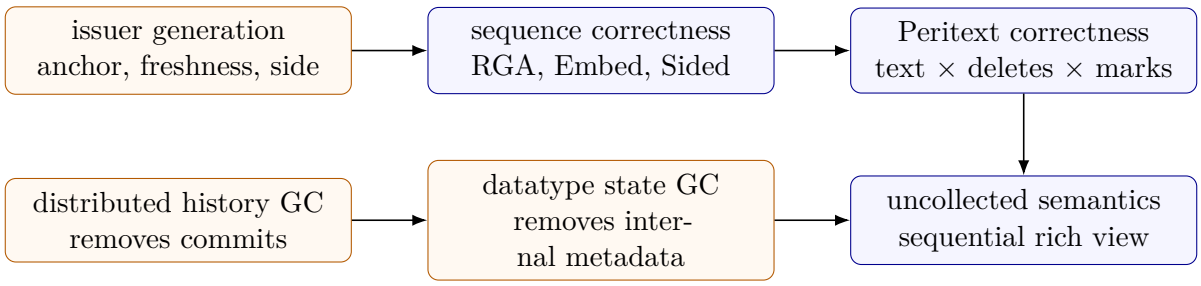


Figure 1: Two evidence paths meet at the user-visible result. The upper path proves the datatype; the lower path erases both collectors to that verified semantics. Blue boxes are correctness results and orange boxes are supplied protocol certificates.

The artifact establishes the following claims.

1. Every generation-certified ordinary or virtual-merge-base execution of each sequence kernel is replication-aware linearizable.
2. Every replicated version in those executions admits a legal witness in an independent sequential specification. All three sequence kernels use an ordinary list as their public sequential state; Peritext uses a list-based rich editor and proves exact rendering.
3. Fugue has the stated forward non-interleaving property but not maximal backward non-interleaving. The FugueMax variant satisfies both directions and is replication-aware linearizable.
4. Sided Peritext has a physical state-GC transition system whose visible trace refines the uncollected semantic system. This protocol composes with the generic asynchronous commit-history collector.

5. The JavaScript runtime is a live, hand-written implementation validated by directed, randomized, snapshot, GC, and differential tests. It is not code extracted from Lean; an evidence manifest records the Lean package and correspondence status of each released datatype.
6. The memory that all three kernels keep about deleted elements is necessary. Self-contained arguments over finite executions show that a design whose state forgets a deleted element reorders survivors on one replica, admits no merge implementing the RGA order, violates the strong list specification, and must retain a logarithmic number of bits per lone survivor.

We use three evidence words consistently. A Lean declaration is *machine-checked*; a JavaScript test is *validated*; a benchmark number is *measured*.

2 Why plain RGA is the motivating example

An insert operation names an anchor and a globally fresh identifier:

$$\mathsf{ins}(x, a) \qquad \text{“insert identifier } x \text{ after identifier } a\text{”}.$$

Here x is the insertion timestamp itself; the RGA model does not carry a second copy of that identifier. A delete names the identifier to hide, $\mathsf{del}(x)$. Identifier 0 is the permanent logical start anchor. A legal issuer may insert after a only when it has already observed a and a is still live, and may delete only a known live identifier.

These premises are not cosmetic. The raw MRDT update is total and can store an insert record with an absent anchor. The user-facing theorem concerns traces minted by the editor protocol. The RGA generation contract records precisely that boundary:

$$\begin{aligned} \mathsf{applicable}(\mathsf{ins}(x, a), S) &\implies (a = 0 \vee a \in \mathsf{live}(S)) \wedge x \text{ fresh}, \\ \mathsf{applicable}(\mathsf{del}(x), S) &\implies x \in \mathsf{live}(S). \end{aligned}$$

The checked bridge turns these issuer-head checks into the history facts used by the public ordinary-list proof. RGA's all-context Join theorem does not use this premise. EmbedRGA and SidedEmbedRGA additionally use their corresponding issuance facts to establish the replay-context condition of their Join theorems.

2.1 RGA

The paper-level state of RGA [2] is a pair $S = (A, G)$:

$$A \subseteq \mathbb{N}^2, \quad (x, a) \in A \text{ records the birth of } x \text{ after } a; \qquad G \subseteq \mathbb{N} \text{ records deleted identifiers.}$$

Both components grow monotonically.

$$\begin{aligned} \mathsf{do}((A, G), \mathsf{ins}(x, a)) &= (A \cup \{(x, a)\}, G), \\ \mathsf{do}((A, G), \mathsf{del}(x)) &= (A, G \cup \{x\}), \\ \mathsf{merge}((A_0, G_0), (A_1, G_1), (A_2, G_2)) &= (A_0 \cup A_1 \cup A_2,\ G_0 \cup G_1 \cup G_2). \end{aligned}$$

To read the state, sort births by timestamp, splice each identifier after its anchor, and filter identifiers in G:

$$\mathsf{seq}(A, G) = \mathsf{filter}_{x \notin G}(\mathsf{foldl}\ \mathsf{insertAfter}\ []\ \mathsf{sort}(A)).$$

These equations are the paper-level form of `Instances.RGA.RGAM` and `Instances.RGA.sequence`. The auxiliary machine `Instances.RGA.birthGraveMachine` stores finite birth and grave sets and

bridges the functional implementation state to a finite proof model. It is not the client specification. The public `Instances.RGA.listSpec` has an ordinary-list state, an abstract legality predicate, and the list-valued query used by `Instances.RGA.verified`.

The state is simple enough to explain both collectors. Deleting the entire document leaves all birth records and graves in the datatype state. Separately, the version DAG still contains every apply and merge commit. Commit-history GC cannot delete RGA tombstones because they live inside a retained state. Datatype-state GC cannot delete old versions because it sees only one materialization. Both are necessary.

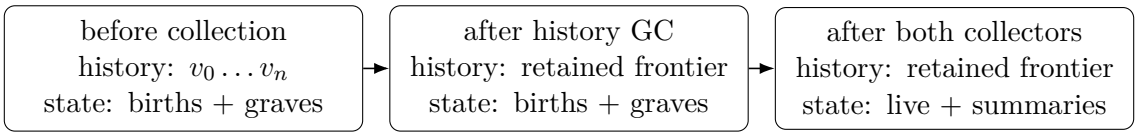


Figure 2: The collectors remove different metadata. Neither subsumes the other: the first changes the commit carrier; the second changes a retained version's materialization. The left arrow is commit-history GC; the right arrow is datatype-state GC.

3 Why sequence state must remember deleted elements

The baseline of Section 2.1 keeps a grave for every deleted identifier. EmbedRGA and SidedEmbedRGA keep no graves, but the coordinate of a live character begins with the codes of its dead ancestors, and the Peritext product keeps a deleted-identifier set. Section 10 measures what this memory costs. This section shows that some such memory is forced. A sequence design whose state forgets a deleted element reorders survivors on a single replica, admits no correct merge, and violates the strong list specification of Attiya et al. [1]. A lone survivor must carry a logarithmic number of bits about the elements deleted around it.

The arguments are elementary. Each is a finite execution that the reader can replay by hand from the definitions below, and none depends on the Lean development or on any representation beyond the properties stated. The section closes by explaining why convergence proofs cannot detect these failures.

3.1 Designs, diamonds, and specifications

Definition 3.1 (sequence design)**.** A sequence design is a tuple $D = (\Sigma, \sigma_0, \mathsf{do}, \mathsf{read}, \mathsf{merge})$ with

$$\mathsf{do} : \Sigma \times Op \to \Sigma, \qquad \mathsf{read} : \Sigma \to Id^*, \qquad \mathsf{merge} : \Sigma \times \Sigma \times \Sigma \to \Sigma,$$

where an operation is $\mathsf{ins}(x, a)$ or $\mathsf{del}(x)$ as in Section 2. A history h is a list of operations and $\mathsf{fold}(\sigma, h)$ applies it left to right. A history is *legal* from σ when each insert uses a fresh identifier and an anchor that is 0 or live in the state it is applied to, and each delete names a live identifier. Legality is the issuer discipline of Section 2; every history below is legal.

Definition 3.2 (diamond)**.** A diamond is a triple of histories (h_L, h_A, h_B) with

$$L = \mathsf{fold}(\sigma_0, h_L), \qquad S_A = \mathsf{fold}(L, h_A), \qquad S_B = \mathsf{fold}(L, h_B), \qquad M = \mathsf{merge}(L, S_A, S_B).$$

Its *observed reads* are read of every state along the three folds, including L, S_A, S_B, and of M. A diamond is one merge step of the version-DAG semantics with L the greatest common ancestor.

Definition 3.3 (sequential specification)**.** The naive list semantics $\mathsf{naive}(h)$ inserts x immediately after a (at the front when $a = 0$) and removes the deleted element. A design is *sequentially sound* when $\mathsf{read}(\mathsf{fold}(\sigma_0, h)) = \mathsf{naive}(h)$ for every legal history. It is *delete-order preserving* when, for every reachable σ and live x, $\mathsf{read}(\mathsf{do}(\sigma, \mathsf{del}(x)))$ is $\mathsf{read}(\sigma)$ with x removed. Sequential soundness implies delete-order preservation.

Definition 3.4 (strong list specification)**.** A diamond satisfies the strong list specification when every observed read contains exactly the identifiers inserted in that observation's causal past and not deleted there, and there is a strict total order $<$ on every identifier inserted in the diamond such that every observed read is sorted by $<$. Thus an order in which two elements have been displayed anywhere is never reversed anywhere, even transitively through elements that have since been deleted.

This is the membership-and-global-order fragment of Attiya et al.'s specification; sequential soundness below supplies the insertion-position condition for the local histories. It mentions no implementation, anchor, or timestamp. The RGA order $\mathsf{seq}(A, G)$ of Section 2.1 is a particular stronger intent: it fixes the merged order completely from births and graves.

3.2 A forgetful delete reorders survivors on one replica

Consider the most direct tombstone-free variant of RGA, the *splice* design. Its state is a forest of live records (t, x, a) whose anchor a is 0 or a live identifier. Insert adds a record. Delete removes the target's record and re-anchors the target's children to the target's own anchor. The read is the RGA traversal of the live forest: children of each node in decreasing identifier order, depth first. No deleted identifier survives in the state.

Proposition 3.5 (delete-order violation)**.** *The splice design is not delete-order preserving, and hence not sequentially sound.*

Proof. Take $h = [\mathsf{ins}(1, 0, a),\ \mathsf{ins}(2, 0, b),\ \mathsf{ins}(3, 1, c)]$. The naive semantics gives $[a]$, then $[b, a]$, then $[b, a, c]$. The splice forest has root children 2 and 1 and child 3 under 1; its read is $[b, a, c]$ as well. Now apply $\mathsf{del}(1)$. The naive result is $[b, c]$. The splice removes record 1 and re-anchors 3 to the root, so the root children are 3 and 2, read in decreasing identifier order as $[c, b]$. The surviving pair b, c has been reversed by a delete that named neither of them. □

Proposition 3.6 (the baseline preserves survivor order)**.** *The RGA baseline of Section 2.1 is delete-order preserving.*

Proof. $\mathsf{del}(x)$ changes only G. By definition $\mathsf{seq}(A, G \cup \{x\}) = \mathsf{filter}_{y \notin G \cup \{x\}}(\ldots)$ is $\mathsf{seq}(A, G)$ with x removed, because the splice fold that precedes the filter reads A alone. □

The grave is exactly the memory that Proposition 3.5 lacks: the dead anchor 1 keeps holding c's place. One might hope to recover this without graves by a smarter re-anchoring rule. The next proposition shows that no local rule makes insert and delete commute, which is the property the baseline's convergence proof rests on (all its operation pairs commute; see Section 4.3).

Proposition 3.7 (no local re-anchoring rule commutes)**.** *Let D be any design over live forests in which $\mathsf{ins}(x, a)$ stores the record $(x, f(\sigma, a))$ with $f(\sigma, a) = a$ whenever a is live, and $\mathsf{del}(x)$ removes x and re-anchors its children to $g(\sigma, x)$ with $g(\sigma, x)$ equal to x's anchor whenever x and that anchor are live, where f and g read only live records. Then there is a reachable state on which an insert after x and the deletion of x do not commute.*

Proof. Let σ_1 contain live records $(1, \cdot, 0)$, $(4, \cdot, 0)$, and $(5, \cdot, 1)$, and let σ_2 be the same with $(5, \cdot, 4)$ instead. Element 5 is a leaf in both. Deleting 5 re-anchors nothing, so $\mathsf{do}(\sigma_1, \mathsf{del}(5)) = \mathsf{do}(\sigma_2, \mathsf{del}(5)) = \rho$, the forest with records 1 and 4 only. Applying $\mathsf{ins}(7, 5, y)$ to ρ stores anchor $v = f(\rho, 5)$, the same value in both cases because ρ is the same state. In the other order, $\mathsf{ins}(7, 5, y)$ finds 5 live and stores $(7, y, 5)$; the subsequent $\mathsf{del}(5)$ re-anchors 7 to 5's anchor, which is 1 from σ_1 and 4 from σ_2. Commutation on σ_1 forces $v = 1$ and commutation on σ_2 forces $v = 4$. □

The proof uses three facts only: erasing a leaf leaves no trace, so the two post-delete states coincide; a prefix-free insert must recompute its anchor from that erased state; and commutation is demanded on all states. Carrying the anchor's ancestor path inside the operation removes the second fact, which is what EmbedRGA's coordinates do.

3.3 No merge on forgetful states implements the RGA order

The single-replica failure above could be blamed on the newest-first tie rule. The next result blames nothing but forgetting. It exhibits two diamonds whose greatest common ancestor and second branch are identical and whose first branches reach the same state, yet whose intended merged reads differ. Every merge function then fails on one of them.

Theorem 3.8 (fooling pair for the RGA order)**.** *Let* $h_L = [\mathsf{ins}(1, 0, a)]$ *and* $h_B = [\mathsf{ins}(2, 0, c)]$*, and let*

$$h_A^1 = [\mathsf{ins}(5, 1, b),\ \mathsf{del}(1)], \qquad h_A^2 = [\mathsf{del}(1),\ \mathsf{ins}(5, 0, b)].$$

Both diamonds are legal. Their RGA orders are $[c, b]$ *and* $[b, c]$ *respectively. Hence any design* D *with* $\mathsf{fold}(L, h_A^1) = \mathsf{fold}(L, h_A^2)$ *produces a merged read that differs from the RGA order on at least one of the two diamonds. The splice design is such a* D*.*

Proof. Legality: a is live when b is anchored at it, 5 and 2 are fresh, and every delete names a live element. For the RGA orders, apply $\mathsf{seq}(A, G)$ of Section 2.1 to the union of births and graves. In the first diamond $A = \{(1, 0, a), (2, 0, c), (5, 1, b)\}$ and $G = \{1\}$. Sorting births by timestamp and splicing gives $[a]$, then $[c, a]$, then $[c, a, b]$; filtering G gives $[c, b]$. In the second diamond $A = \{(1, 0, a), (2, 0, c), (5, 0, b)\}$ and $G = \{1\}$: $[a]$, then $[c, a]$, then $[b, c, a]$; filtering gives $[b, c]$. In both diamonds L and S_B are literally the same states, so if S_A also coincides then $\mathsf{merge}(L, S_A, S_B)$ is one state with one read, which cannot equal both $[c, b]$ and $[b, c]$. For the splice design, h_A^1 stores $(5, b, 1)$ and then re-anchors it to 0 on deleting 1, while h_A^2 stores $(5, b, 0)$ directly; both yield the single record $(5, b, 0)$. □

The information the splice destroys is precise: whether b occupies the slot of the dead a or a slot of its own. The baseline keeps it as the birth record $(5, 1, b)$ together with grave 1. EmbedRGA keeps it because b's coordinate in the first diamond begins with a's code and in the second is a root code. In both cases the two branch states differ, and the theorem does not apply.

3.4 The strong list specification forces the same memory

Theorem 3.8 is relative to RGA's intended order. One could object that a different intent would rescue forgetting. The strong list specification refutes this objection: it is implementation independent and still separates two diamonds with coinciding forgetful states. The witness needs five elements because the constraint must pass transitively through elements that have been deleted.

The sequential-soundness premise in the next two theorems is load-bearing: it fixes the observed orders before the merge. The global-order condition by itself would not determine those local reads.

Theorem 3.9 (fooling pair for the strong list specification)**.** *Let* $h_L = [\mathsf{ins}(1,0,m),\ \mathsf{ins}(2,0,g)]$*, so* L *reads* $[g,m]$*, and let* $h_B = [\mathsf{ins}(9,2,y)]$*, so* S_B *reads* $[g,y,m]$*. Let*

$$h_A^1 = [\mathsf{ins}(5,0,x),\ \mathsf{del}(2),\ \mathsf{del}(1)], \qquad h_A^2 = [\mathsf{ins}(5,1,x),\ \mathsf{del}(1),\ \mathsf{del}(2)].$$

Both diamonds are legal, and in both the final read of the first branch is $[x]$*. For a sequentially sound design, the strong list specification requires the merged read* $[x,y]$ *in the first diamond and* $[y,x]$ *in the second. Hence any sequentially sound design with* $\mathsf{fold}(L,h_A^1) = \mathsf{fold}(L,h_A^2)$ *violates the strong list specification on one of them.*

Proof. Sequential soundness fixes the observed reads of the three folds to their naive values. The first branch reads $[x,g,m]$, $[x,m]$, $[x]$ in the first diamond and $[g,m,x]$, $[g,x]$, $[x]$ in the second. In the first diamond $x < g$ was displayed on the first branch and $g < y$ on the second, so any total order consistent with all reads has $x < y$; the merged read, a sequence over $\{x,y\}$, must be $[x,y]$. In the second diamond $y < m$ was displayed on the second branch and $m < x$ on the first, so $y < x$ and the merged read must be $[y,x]$. The two diamonds share L and S_B; if they also share S_A, the merge has one output. Note that x and y are never displayed together before the merge: the obligation is carried entirely by the deleted g and m. □

The theorem says what a correct state must contain after h_A^1 and h_A^2: two different values, although both documents consist of the single character x. A design whose state for a one-character document is a function of that character alone cannot satisfy the strong list specification, whatever its merge.

3.5 How much must be remembered

The five-element witness generalizes to a lower bound on the information a survivor must retain about the elements deleted around it.

Theorem 3.10 (retention lower bound)**.** *Let* D *be sequentially sound and satisfy the strong list specification on every legal diamond. For every* $n \geq 1$ *there is a legal history* h_L *inserting* n *elements and legal continuations* $h_A^0, \ldots, h_A^n$*, each inserting one element* x *and then deleting all* n *elements of* h_L*, such that the states* $\mathsf{fold}(L,h_A^k)$ *take at least* $\lceil (n+1)/2 \rceil$ *distinct values. Every one of these states reads* $[x]$*. Consequently a state whose document is the single character* x *must carry at least* $\log_2 \lceil (n+1)/2 \rceil$ *bits beyond* x*'s identifier and payload once* n *elements have been deleted.*

Proof. Let h_L insert $e_1, \ldots, e_n$ each immediately after the previous one, so L reads $[e_1, \ldots, e_n]$. Write $e_0 = 0$ and let $h_A^k = [\mathsf{ins}(x, e_k, \cdot),\ \mathsf{del}(e_1), \ldots, \mathsf{del}(e_n)]$, whose first read is $[e_1, \ldots, e_k, x, e_{k+1}, \ldots, e_n]$ and whose last is $[x]$. Fix $k < k'$ with $k' - k \geq 2$ and choose g with $k < g < k'$. Let $h_B = [\mathsf{ins}(y, e_g, \cdot)]$, so S_B reads $[e_1, \ldots, e_g, y, e_{g+1}, \ldots, e_n]$. In the diamond with h_A^k, the first branch displayed x before e_{k+1}, and e_{k+1} is displayed at or before e_g, which the second branch displayed before y; so $x < y$ and the merged read must be $[x,y]$. In the diamond with $h_A^{k'}$, the second branch displayed y before e_{g+1}, and the first branch displayed e_{g+1} at or before $e_{k'}$, which precedes x; so $y < x$ and the merged read must be $[y,x]$. The two diamonds share L and S_B, so $\mathsf{fold}(L,h_A^k) \neq \mathsf{fold}(L,h_A^{k'})$, otherwise the single merge output would violate one of them. The indices $0, 2, 4, \ldots$ are pairwise at distance at least 2, giving $\lceil (n+1)/2 \rceil$ pairwise distinct states. The bit bound follows because these states share identifier and payload of x and differ only in retained information. □

The bound concerns information, not representation, and the three kernels pay it in three currencies. The baseline pays with graves: n identifiers, shared by every survivor. EmbedRGA pays inside the survivor: x's coordinate under h_A^k is the code of e_k's path followed by $\Gamma(t_x - e_k)$, different for different k, and as long as the ancestor spine. The ancestor-spine workload in Section 10 is this bound made concrete. Peritext pays a third way for its marks, retaining deleted identifiers whose positions mark boundaries still need. What the bound excludes is a design whose state after n deletions is the live document plus a bounded number of bits. Attiya et al. [1] introduced the full strong and weak list specifications and proved an $\Omega(D)$ metadata lower bound for push-based protocols with at least three replicas, including executions with causal atomic broadcast. Their protocol model and bound are not the theorem above; the diamond argument isolates a weaker logarithmic consequence for ternary merge using three replicas and one merge.

3.6 Convergence does not force retention

Nothing in Proposition 3.5 involves a merge. The reorder happens on one replica, inside do, so a design that reorders the same way on every replica converges while violating its sequential specification. Replication-aware linearizability certifies convergence to the datatype's own fold; when the fold itself reorders survivors, every replica agrees on the wrong answer and the convergence theorem is satisfied. This is why Table 2 lists sequential reference semantics separately from replication-aware linearizability, and why Section 4.3 treats the sequential theorem as independent evidence. The memory retained by the three kernels is justified by Propositions 3.5 and 3.7 and Theorems 3.8, 3.9, and 3.10, not by convergence.

4 Three verified sequence representations

The three representations implement the same insert-after/delete interface, but they retain different information and offer different ordering policies:

Kernel	Position of a live descendant	What survives deletion of its anchor
RGA	identity edge to anchor a	birth of a and a grave marker
EmbedRGA	immutable anonymous path coordinate	path geometry; identity a may disappear
SidedEmbedRGA	path coordinate with an L/R band	path geometry and compact mint-policy summary

4.1 EmbedRGA

EmbedRGA assigns an immutable coordinate when an operation is minted. An insert carries a prefix π derived from its observed anchor and appends an ordered prefix-code word:

$$\mathsf{coord}(\mathsf{ins}(t, \pi, a, x)) = \pi \cdot \Gamma(t - a).$$

The materialized state is one list of (t, x, coord) records strictly sorted by coordinate. Insert is sorted insertion, delete filters one record, and ternary merge applies OR-set survival relative to the LCA before re-sorting. Because distinct honest births have distinct keys, sorted lists with the same record set are equal. This canonical representation is the core of the convergence proof.

The coordinate carries path geometry without retaining each deleted ancestor identity. This is more than a constant-factor encoding change after state GC: an ancestor spine can leave thousands of identity-bearing tombstones in RGA, whereas an embedded descendant needs only its path code.

4.2 SidedEmbedRGA and the Fugue policies

SidedEmbedRGA adds $d \in \{L, R\}$ to a minted insert and encodes it in a sided block:

$$\mathsf{scoord}(\mathsf{ins}(t, \pi, a, d, x)) = \pi \cdot \mathsf{sBlock}_\Gamma(d, t - a).$$

The MRDT effect treats the side as immutable operation data. A generation policy decides which side and parent to mint from the issuer's knowledge. This separation lets one proved kernel host several policies.

Definition 4.1 (non-interleaving)**.** A local typing run is a sequence of causally consecutive inserts made by one replica. A merged read is forward non-interleaving when each forward run forms one interval. The maximal property also protects the paper's backward-run cases, subject to its stated exception.

The Lean development proves:

$$\begin{aligned} \mathsf{FugueReach} &\implies \mathsf{ForwardNI}, \\ \mathsf{Fugue} &\not\implies \mathsf{MaximalBackwardNI}, \\ \mathsf{FugueMaxReach} &\implies \mathsf{ForwardNI} \wedge \mathsf{BackwardNI} \wedge \mathsf{MaximalNI}. \end{aligned}$$

The corresponding Lean results are:

`fugue_forward_ni`, `fugue_not_maximally_noninterleaving_backward`, `fuguemax_forward_ni`, `fuguemax_backward_ni`, `fuguemax_maximally_noninterleaving`.

The coordinate-level `FMSig` remains an internal replay model. The separate enriched `FugueMax.datatype` stores the birth/origin metadata needed by the exact positional issuer and has its own public package, `FugueMax.verified`. Its sequential state is a plain list. For ordinary and virtual certified executions, `FugueMax.correct` gives one witness for list correctness and all three maximal-non-interleaving clauses. It does not assume a separate `MaxReach` proof. This package is distinct from `Instances.ProductionRGA.sided`; its datatype-state collection remains staged, and no independent FugueMax runtime is claimed.

4.3 What is proved for all three

Table 1 separates merge preservation from the operation-issuance premise of the public theorem. "Join for all contexts" means `Join`; "Join under G" means `JoinOn` with replay-context predicate G.

RDT	Merge theorem	Issuance policy	Role of issuance
RGA	Join for all contexts	`applicable`	list legality
EmbedRGA	Join under `EHonestCore`	`eApplicable`	Join condition and list legality
SidedEmbedRGA	Join under `SHonestCore`	`sApplicable`	Join condition and list legality

Table 1: Merge scope and issuance are independent proof axes. RGA issuance is load-bearing for its intended list specification even though its merge theorem holds at every replay context.

For each kernel, the repository packages a generation contract, ordinary and virtual-merge-base convergence, an independent list specification, and the bridge from certified minting to a legal list witness. The capstones are `Instances.RGA.verified`, `Instances.ProductionRGA.embed`, and `Instances.ProductionRGA.sided`. Each is a public `VerifiedMRDT` package. The repository retains the separately named `replayEmbed` and `replaySided` packages for raw-fold proofs.

The sequential theorem matters independently of convergence. EmbedRGA and SidedEmbedRGA refine explicit list machines for a legal minted linear history. Each kernel also has the merged-history `VerifiedMRDT` theorem: each replicated version has an exact, causally ordered, legal sequential witness with the intended list view. Each package supplies one `EventOrder` stored in `VerifiedMRDT.replayOrder`; write ord for its directed full-event relation. `loOn` retains a visibility edge when ord relates its events in either direction, and uses the selected direction for concurrent pairs subject to its absorber condition. The submission manuscript reserves rc for an operation-level conflict relation and its event lift; the independently selected event order here need not be that lift. The proof-level `UpdateSig` is derived from the MRDT signature. The generation premise preserves the actual editor protocol rather than silently totalizing invalid anchor use as a no-op.

5 Peritext over the sided sequence

Peritext represents formatting as immutable mark events over a character sequence. The production-facing Lean state has three logical components:

$$S_{rich} = \underbrace{S_{text}}_{\text{sided insertion shadow}} \times \underbrace{D}_{\text{grow-only deleted character IDs}} \times \underbrace{M}_{\text{grow-only mark events}}.$$

Native text deletion is disabled. A user deletion adds the identifier to D, preserving the birth position needed to resolve mark boundaries. A mark records a globally fresh mark ID, type, value, start and end identifiers, and boundary sides.

At the paper level, a rich operation is one of

$$o ::= \mathsf{textIns}(t, \pi, a, d, x) \mid \mathsf{delete}(x) \mid \mathsf{mark}(m, \mathit{kind}, \mathit{value}, a_s, d_s, a_e, d_e, \mathit{action}).$$

The update routes the operation to exactly one product component:

$$\begin{aligned}(T, D, M) + \mathsf{textIns}(\cdots) &= (T + \mathsf{ins}(\cdots), D, M),\\ (T, D, M) + \mathsf{delete}(x) &= (T, D \cup \{x\}, M),\\ (T, D, M) + \mathsf{mark}(m, \ldots) &= (T, D, M \cup \{\mathsf{mark}(m, \ldots)\}).\end{aligned}$$

Ternary merge is the product of the sided-text merge and two set unions. This product structure explains convergence, but not client legality: the cross-component issuance policy supplies that missing relation.

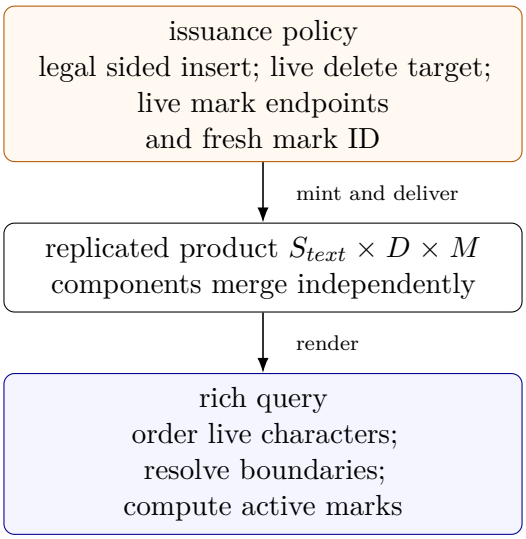


Figure 3: Peritext state and query pipeline. Text ordering, logical deletion, and mark evidence merge independently. The rich query is the only place that combines all three components.

The cross-component issuance policy in Figure 3 requires an insert to satisfy the sided sequence policy, a deletion to name a known live character, and a mark to name known live endpoints with a fresh mark ID. The theorem `coreHonest_of_mint` derives the sequence history fact needed by the product Join proof.

The independent rich sequential machine stores an ordinary list of character records, a deleted-ID set, and a mark-event set. The representation relation equates the sided MRDT projection with that list after removing the logical root, and equates both grow-only stores exactly. Consequently:

Theorem 5.1 (rich sequential correctness)**.** *For every legal minted operation list ρ, the state obtained by folding the Sided Peritext MRDT is related to the state obtained by running the independent rich editor machine on ρ.* `rich_sequentially_correct`

The production signature narrows the query to rendered rich text without changing the operational carrier. `richVerified` transfers the merged-history legalization to this rendered query boundary. It proves query agreement for the document-order rich-text render, not only for the three raw components. `richReplayVerified` remains as the explicitly internal raw-fold package. The JavaScript export `peritext` selects `peritextSidedEmbedRGA`; the RGA and one-sided EmbedRGA variants remain available as controlled baselines.

6 Datatype-state garbage collection

Peritext cannot discard every deleted birth immediately. Mark endpoints and future Fugue minting can still depend on old order. A compact materialization therefore contains

$$C = \langle T, K, D, M, c \rangle,$$

where T is retained sided text, K is a finite LiveGap policy summary, D is retained deletion evidence, M is the mark list, and c is a stable Lamport cutoff. The cutoff has one interpretation:

$$\mathsf{keep}(x) = \mathsf{false} \quad \Longrightarrow \quad x \leq c.$$

It is not a second tombstone set. It compactly separates collected old identifiers from future fresh identifiers. A new Lamport identifier must be strictly greater than c, so it is necessarily retained by the epoch policy.

The refinement relation connects this compact state to the uncollected rich state $F = (T_F, D_F, M_F)$. It existentially records a retention predicate k and Fugue knowledge K_F and requires

$$\begin{aligned} T_F = \mathsf{gFold}(K_F), \qquad & T = \mathsf{filter}_k(T_F), \\ \mathsf{GapMapOK}(K_F, K), \neg k(x) \qquad & \Rightarrow x \le c, \\ \forall q.\ \mathsf{query}(C, q) = \mathsf{render}(F, q). & \end{aligned}$$

This is `StateGC.Protocol.Represents`. Equality of current reads is only one field. The text projection and gap-map fields are what make subsequent insertion and merge provable.

6.1 Three atomic collectors

$$\frac{\forall x.\ \neg k(x) \Rightarrow x \in D \quad \mathsf{endpoints}(M) \subseteq k \quad K' \text{ exactly summarizes future mint choices}}{\langle T, K, D, M, c\rangle \rightsquigarrow \langle \mathsf{filter}_k(T), K', D, M, \max(c, c')\rangle}\ \textsc{Collect-Text}$$

$$\frac{}{\langle T, K, D, M, c\rangle \rightsquigarrow \langle T, K, D \cap \mathsf{ids}(T), M, c\rangle}\ \textsc{Trim-Deletes}$$

$$\frac{m = \mathsf{add} \quad r = \mathsf{remove} \quad m < r \quad \alpha(m, r, M) \quad \beta(m, r, T)}{\langle T, K, D, M, c\rangle \rightsquigarrow \langle T, K, D, M \setminus \{m, r\}, c\rangle}\ \textsc{Drop-Mark-Pair}$$

Figure 4: The three Peritext state-GC steps. The α and β premises exclude delayed same-type marks and characters in the Lamport growth window; checked counterexamples show that both are necessary.

Each rule in Figure 4 preserves every rendered query: `collectText_query_preserved`, `trimDeleted_query_preserved`, and `dropMarkPair_query_preserved`. Query preservation alone is insufficient: the compact state must accept future operations and later merge with replicas collected in other epochs. The continuation proof uses three additional facts.

1. The LiveGap map reconstructs exactly the Fugue side, parent, and parent chain used to mint the next insert (`compactInsertOp_exact`).
2. Future Lamport IDs lie above the stable cutoff and therefore cannot be mistaken for collected IDs (`TextPlan.keeps_fresh`).
3. Independently collected branches translate to a common projection before ternary merge; compact merge equals that projection of semantic merge (`merge_text_after_epoch_translation`).

6.2 Protocol refinement

A physical version may have no materialization, but every replica head has one. Each materialized head carries a `Represents` witness linking compact text, gap summaries, cutoff, and all rich queries to the semantic state. Collection and epoch reframing are silent. Ordinary apply/merge and virtual merge are visible.

The physical state contains one ghost semantic configuration for the proof and a partial materialization map:

$$P = \langle C_{sem}, \mathsf{compact} : \mathit{Version} \rightharpoonup \mathsf{CompactState} \rangle.$$

Every mapped version must represent its semantic state, and every replica head must be mapped. The transition labels are optional semantic labels:

$$\begin{array}{lll} P \xrightarrow{\tau} P' & : & \mathsf{collectText},\ \mathsf{trimDeleted},\ \mathsf{dropMarkPair},\ \mathsf{reframe}, \quad C'_{sem} = C_{sem}, \\ P \xrightarrow{\ell} P' & : & \mathsf{ordinary},\ \mathsf{virtual}, \qquad\qquad\qquad\qquad\quad C_{sem} \xrightarrow{\ell}_{\mathsf{StepV}} C'_{sem}. \end{array}$$

An apply label additionally requires its timestamp to lie above the compact head's stable cutoff. Fetch advances the Lamport clock before the runtime may mint that operation.

Theorem 6.1 (Peritext state-GC refinement)**.** *Erase collection and reframe labels from any finite valid physical execution. The remaining labels form an execution of the uncollected virtual-merge-base semantics, with the same rich queries at materialized heads.* `StateGC.Protocol.refines`

7 Distributed commit-history garbage collection

The second collector is generic to the MRDT runtime. Replica r stores only

$$L_r = \langle h_r, C_r \rangle, \qquad h_r \in C_r,$$

where h_r is its head and C_r its locally held immutable commits. A remote fetch unions advertised commits; a separate head-sync action selects a held version. Evidence is derived from authored commits already in C_r.

The asynchronous transitions are:

$$\frac{M = C_s}{W \longrightarrow W[d \mapsto \langle h_d, C_d \cup M \rangle]}\ \textsc{Fetch}$$

$$\frac{v \in C_r}{W \longrightarrow W[r \mapsto \langle v, C_r \rangle]}\ \textsc{Head-Sync} \qquad \frac{\mathsf{Certificate}(\langle h_r, C_r \rangle, K)}{W \longrightarrow W[r \mapsto \langle h_r, K \rangle]}\ \textsc{Collect}.$$

Figure 5: Distributed commit-history transitions. Fetch corresponds to receiving immutable commits from a remote; it does not move the local head. Collection changes one replica and requires complete frontier evidence.

A local collection requires frontier evidence for every registered replica and a keep certificate that preserves head reachability and LCA answers. The compressed DAG connects retained versions according to original reachability; it need not retain the old root or every path node.

The collecting protocol refines the same asynchronous fetch/head-sync protocol with GC erased. State GC then composes directly with this refinement:

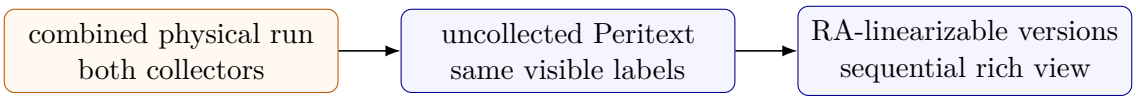


Figure 6: End-to-end semantic path for both collectors. The history collector is framework supplied; the state collector is Peritext supplied. Both arrows denote proved semantic implications: `CombinedSteps.refinesV` followed by the verified Peritext capstones. They do not denote implementation extraction.

`GC.CombinedSteps.refinesV` is the generic composition theorem. It avoids an intermediate global or stop-the-world semantics: distributed GC refines the no-GC asynchronous system directly.

8 Ordering anomalies and evidence

Finite anomaly fixtures are useful regression tests, but they are not a substitute for quantified intent theorems. Table 2 separates the two kinds of evidence.

Property or corpus	Scope	RGA	Embed RGA	Sided/ FugueMax
Sequential reference semantics	all legal sequential histories, Lean	proved	proved	proved
RA-linearizability	certified ordinary and virtual histories, Lean	proved	proved	proved
Forward non-interleaving	all Fugue-policy reachable states, Lean	–	–	proved
Maximal backward non-interleaving	policy-level quantified result, Lean	no claim	no claim	Fugue refuted; FugueMax proved
L1, L3a/b, L4, L5, L7–L13, L16, L17, L19	15 reviewed directed JS fixtures	baseline outputs	baseline outputs	all pass expected reads
L2, L6, L14, L15, L18, L20–L25	no current reviewed runtime fixture	untested	untested	untested
Retention of deleted-element order	forced by Theorems 3.9 and 3.10, pen and paper	graves G	dead codes	dead codes

Table 2: Current anomaly evidence. "Untested" is not reported as a pass. The directed corpus validates implementation behavior; only the quantified Lean rows establish general semantic properties.

The L19 fixture is a backward-run case. Two replicas insert runs toward the front of a shared document. The reviewed sided implementations produce $[50, 30, 10, 61, 41, 21, 1]$, keeping each local run contiguous. The stronger FugueMax theorem generalizes beyond this fixture. We do not claim an external Yjs, Automerge, or Loro anomaly result from the current harness because the repository does not contain reviewed adapters for the full anchored corpus.

9 Implementation

The JavaScript runtime provides content-addressed commits, replicas, fetch and head synchronization, three-way merge, snapshots, distributed frontier evidence, commit GC, and datatype state-GC hooks. Persistent maps and sets avoid copying full logical stores on each update. Binary snapshot codecs pack identifiers and coordinate symbols rather than shipping textual bit strings.

The runtime exposes six relevant datatypes: `rga`, `embedRGA`, `liveGapSidedEmbedRGA`, and Peritext built over each of those three kernels. The default `peritext` is the sided variant. Snapshot recovery means loading a native continuation snapshot into a fresh in-process state and materializing its visible document; it does not create a new logical replica or change replica identity.

The Lean code and JavaScript have different trust boundaries. Lean verifies mathematical definitions and refinements. JavaScript tests check that the implementation agrees with expected reads, a split-map oracle, randomized fork/join executions, snapshot round trips, frontier refusal, offline replica return, and collection invariants. Differential testing is a practical next step toward an extracted reference executable; no Lean-to-C-to-Wasm path is claimed here.

10 Evaluation

All reported cells are generated from checked-in drivers on Apple M4 Pro, 24 GB RAM, macOS arm64, and Node.js 26.3.1. Repeated reports use three isolated processes per cell. Drivers fail before reporting a cell if their semantic, convergence, snapshot, or scenario-specific GC gates fail.

10.1 Sequence kernels

Table 3 reports medians for four real sequential traces. *Recovery* loads the kernel's continuation snapshot into a fresh state and materializes the visible sequence. Snapshot bytes are not a lower bound: each kernel has its own current codec.

Trace	Ops	Kernel	Apply (ms)	Snapshot (B)	Recovery (ms)
friendsforever	35,200	RGA	32.0	145,183	17.4
		Embed	20.5	59,064	13.4
		Sided	34.6	63,113	17.9
clownschool	43,500	RGA	19.1	138,358	16.8
		Embed	16.3	62,412	12.1
		Sided	25.8	62,912	16.0
seph-blog1	368,209	RGA	1,892.5	1,440,927	348.0
		Embed	1,795.1	161,046	47.4
		Sided	1,924.0	210,153	84.3
automerge-paper	260,978	RGA	2,031.9	1,182,880	267.8
		Embed	1,927.2	249,973	88.6
		Sided	2,083.2	323,888	138.6

Table 3: Repeated sequence-kernel comparison.

On `seph-blog1`, EmbedRGA uses 8.9 times fewer snapshot bytes than RGA; on `automerge-paper`, it uses 4.7 times fewer. SidedEmbedRGA uses 1–30% more snapshot space than EmbedRGA across these traces. On the two large traces, its apply time is 7–8% higher. This is the central design tradeoff: EmbedRGA minimizes current continuation state, while sided ordering pays metadata and compute for a stronger interleaving policy.

10.2 Both collectors in Peritext

Table 4 reports the combined-GC mode. Durable state is the continuation state needed for future editing, including the current materialization and GC summaries, not only visible characters.

Workload	Kernel	Time (s)	Durable state	Commits before → after
concurrent rich	Embed	13.07	95,438 B	4,108 → 1
	Sided	14.53	212,685 B	4,108 → 1
format trace	Embed	36.18	72,182 B	26,753 → 1
	Sided	36.63	203,219 B	26,753 → 1
mark churn	Embed	42.21	1,014 B	4,543 → 1
	Sided	42.40	3,764 B	4,543 → 1
empty document	Embed	0.32	9 B	1,203 → 1
	Sided	0.32	61 B	1,203 → 1

Table 4: Repeated Peritext evaluation with both continuation-safe state and commit-history GC.

The RGA rows previously shown here used current-read-only dead-leaf pruning. The negative control in `runtime/test/rga.test.js` concerns the PeritextRGA wrapper: its shadow omits text deletions and thus admits retained anchors. Erasure rejects a continuation admitted by that wrapper. Plain RGA instead rejects insertion after an observed deleted anchor. These are different issuance contracts; the wrapper control is not an impossibility theorem for plain RGA reclamation. Those measurement rows remain omitted. The new Lean RGA packing certificate retains every identifier; it has not yet been implemented or measured in the JavaScript runtime.

The empty-document row is a sanity check, not an asymptotic theorem. On this settled workload, both tombstone-free kernels remove all character identities and marks and history shrinks to one commit. The remaining 9–61 bytes are codec and continuation headers.

An isolated ancestor-spine workload explains why EmbedRGA remains useful even after both collectors. It performs 21,200 text operations and deletes 6,000 ancestor identities. In the historical negative-control run, RGA retains 150,561 bytes and all 6,000 identities; EmbedRGA retains 46,072 bytes and no deleted identity; SidedEmbedRGA retains 80,965 bytes and no deleted identity. All histories shrink to one commit. Anonymous path geometry can therefore be substantially smaller than an identity-bearing ancestor spine; the RGA number is descriptive, not evidence for its unsafe state hook.

10.3 External plain-text systems

Table 5 gives one representative common-interface trace. These systems expose different continuation formats. Save bytes therefore describe each current native continuation snapshot; they are not semantic apples-to-apples lower bounds. We compare rich text only among the three Sal Peritext variants because the harness does not coerce external rich-text semantics into Peritext's mark model.

System (`seph-blog1`)	Total (ms)	Save (B)	Recovery (ms)
Sal RGA	454.89	1,440,927	356.78
Sal EmbedRGA	367.41	161,046	52.67
Sal SidedEmbedRGA	508.94	210,153	85.23
Yjs	750.40	338,289	9.31
Automerge	10,018.46	205,250	443.01
Loro	303.07	336,808	0.23
list-positions	485.01	572,686	3.65

Table 5: Repeated plain-text comparison on a common sequential trace. Native save formats retain different auxiliary information.

The fast recovery values for Yjs and Loro reflect their native snapshot representations and optimized implementations. They do not establish the absence of ordering anomalies, and the current paper does not infer such a result from performance tests.

11 Evidence boundary and limitations

The current artifact proves the semantic kernels, virtual-merge-base executions, issuance policies, sequential refinements, Peritext state GC, distributed commit GC, and their composition. It validates but does not verify the JavaScript transliteration. The retention arguments of Section 3 are pen-and-paper proofs over finite executions and are not part of the Lean ledger. Same-replica crash recovery and durable replica identity are not yet a formal protocol. The reviewed JavaScript anomaly corpus contains 15 directed cases, not all labels L1–L25. External rich-text semantics are deliberately not compared.

The stable-cut theorem assumes Lamport-style monotone identifiers. A runtime must advance its local clock after fetch before minting. The compact LiveGap map is required for future Fugue decisions even when it is irrelevant to the current read. Removing a stable dead leaf without the required policy summary has a checked counterexample that changes a future read.

12 Reproduction

From the repository root, run `./scripts/check-mrdt-refactor.sh` for the Lean and runtime gates and `./scripts/check-working-papers.sh` for both papers and their declaration ledger. Regenerate the measurements with `node benchmarks/tools/run-kernel-comparison.mjs 3` and `node benchmarks/tools/run-peritext-paper.mjs --repeat 3`.

References

[1] Hagit Attiya, Sebastian Burckhardt, Alexey Gotsman, Adam Morrison, Hongseok Yang, and Marek Zawirski. Specification and complexity of collaborative text editing. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing (PODC)*, 2016.

[2] Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. Replicated abstract data types: Building blocks for collaborative applications. *Journal of Parallel and Distributed Computing*, 71(3):354–368, 2011. https://doi.org/10.1016/j.jpdc.2010.12.006.